

Image Recognition



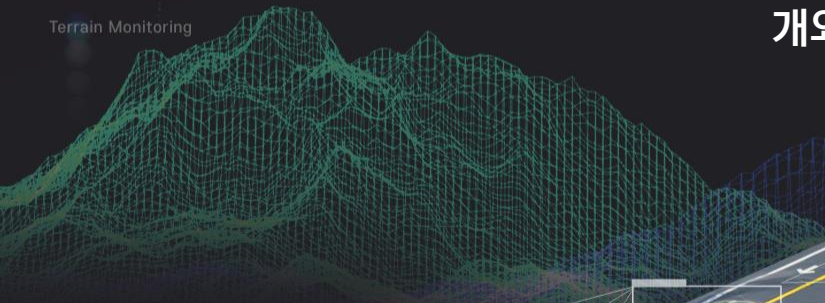
강원혁신플랫폼

컴퓨터 비전



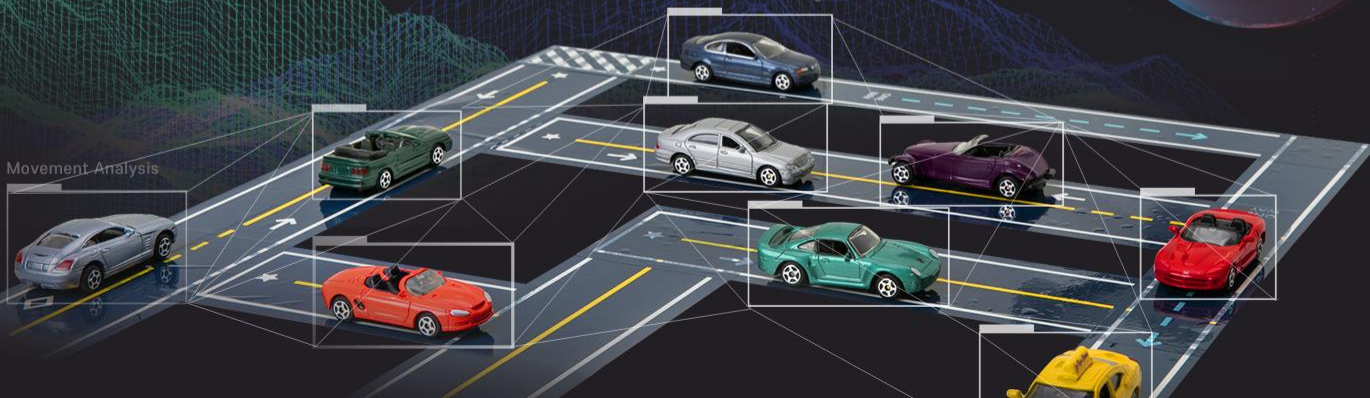
개와 고양이 이진 분류

Terrain Monitoring



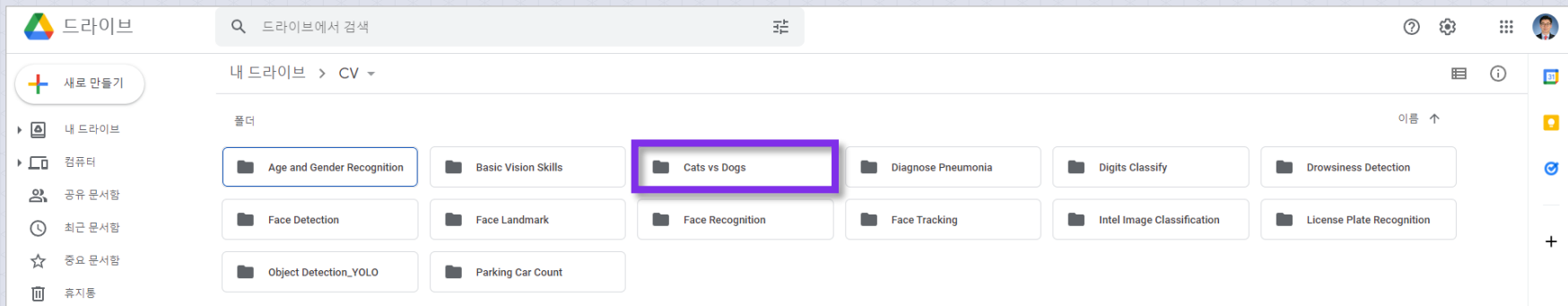
Autonomous Driving

Movement Analysis



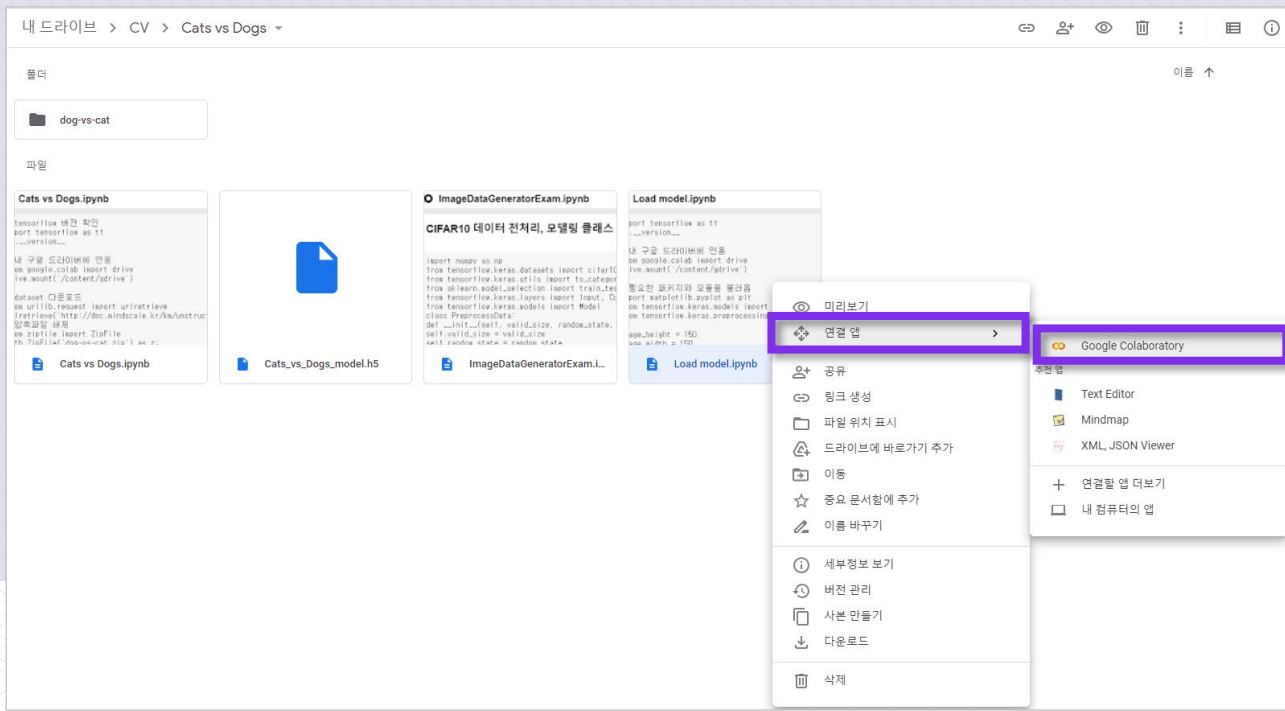
① 실습 파일 복사하기

- 🔄 학습된 모델을 이용해 추론을 수행해보자.
- 🔄 구글 드라이브 > CV > Cats vs Dogs 폴더를 클릭한다.



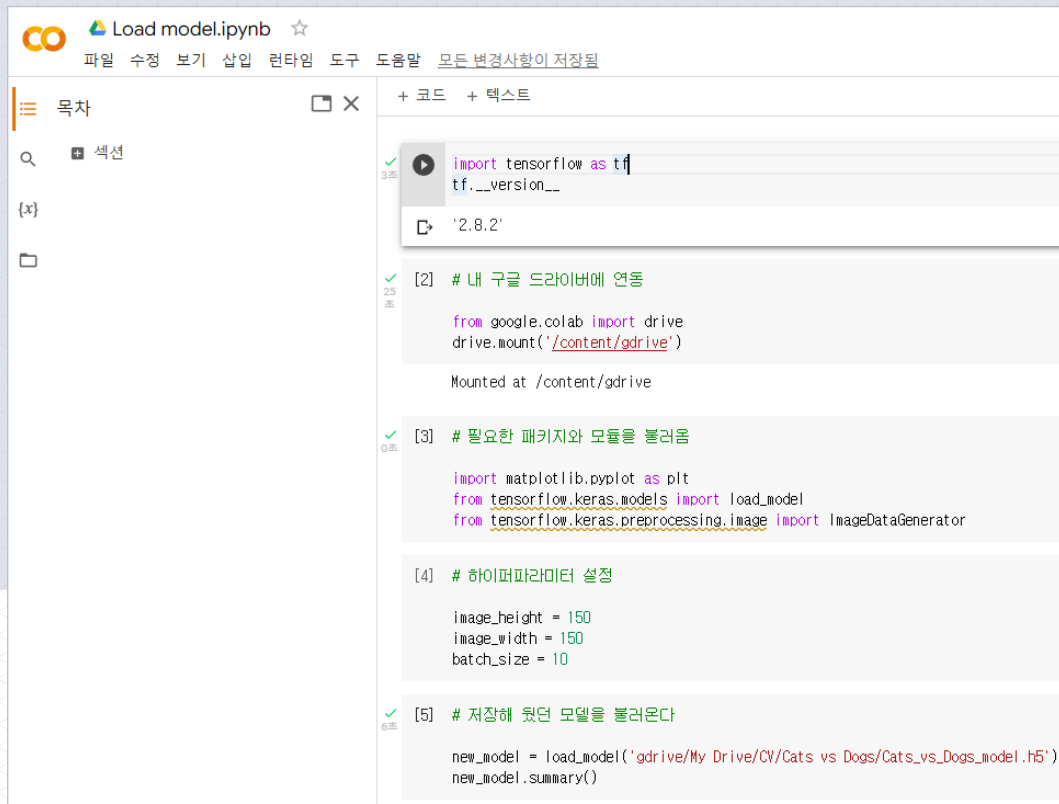
① 실습 파일 복사하기

- ❶ 아래의 그림의 Cats vs Dogs 폴더에서 **Load model.ipynb**를 선택 후
오른쪽 마우스를 클릭하여 연결 앱 > **Google Colaboratory**을 클릭한다.



① 실습 파일 복사하기

아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



```
Load model.ipynb ☆
파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨

+ 코드 + 텍스트

import tensorflow as tf
tf.__version__

'2.8.2'

[2] # 내 구글 드라이브에 연동

from google.colab import drive
drive.mount('/content/gdrive')

Mounted at /content/gdrive

[3] # 필요한 패키지과 모듈을 불러옴

import matplotlib.pyplot as plt
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import ImageDataGenerator

[4] # 하이퍼파라미터 설정

image_height = 150
image_width = 150
batch_size = 10

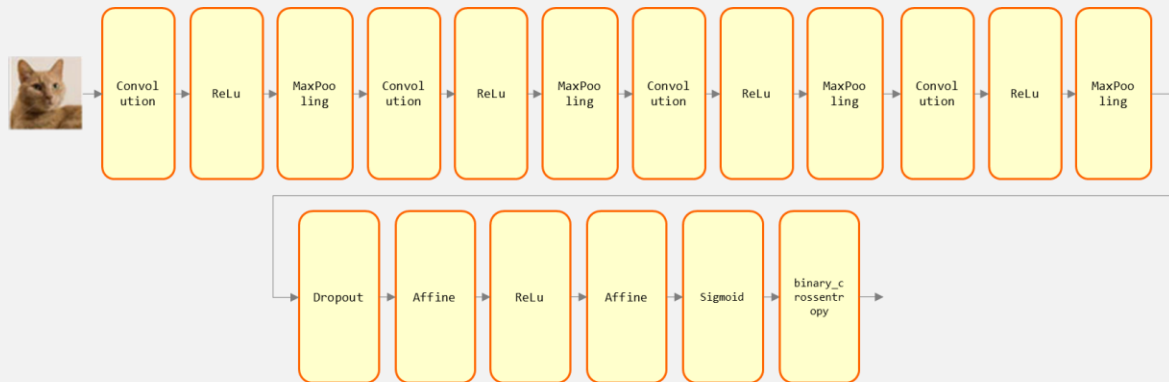
[5] # 저장해 뒀던 모델을 불러온다

new_model = load_model('gdrive/My Drive/CV/Cats vs Dogs/Cats_vs_Dogs_model.h5')
new_model.summary()
```

학습된 인공지능망 모델을 읽어와서 테스트 데이터 셋으로 모델을 평가 및 추론을 수행해보자.

- 테스트 데이터 셋의 16장 이미지로 분류 예측을 수행해 그 결과 이미지를 출력한다.
- 이를 위해 학습된 모델 “Cats_vs_Dogs_model.h5” 파일을 메모리에 읽어온다.
- 즉, 앞에서 생성한 6층 인공지능망 네트워크 구조에 학습된 매개변수로 초기화한다.

6층 인공지능망 모델의 계층구조로 이루어진 네트워크 구조



인공지능망 네트워크 구성

② 구글 드라이브에 연동하기

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 내 구글 드라이브에 연동
```

```
from google.colab import drive
```

```
drive.mount('/content/gdrive')
```

```
Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).
```

② 구글 드라이브에 연동하기

- ① [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택한다.
- ② gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요 **Google Drive에 연결**

Google 계정으로 로그인

계정 선택

Google Drive for desktop(으)로 이동

장원중
wjjang2040@gmail.com

② 다른 계정 사용

계속 진행하기 위해 Google에서 내 이름, 이메일 주소, 언어 환경 설정, 프로필 사진을 Google Drive for desktop과(와) 공유합니다. 앱을 사용하기 전에 Google Drive for desktop의 개인정보처리방침 및 서비스 약관을 검토하세요.

한국어 ▼ 도움말 개인정보처리방침 약관

구글 드라이브 접근 허용

Google 계정으로 로그인

Google Drive for desktop에서 내 Google 계정에 액세스하려고 합니다

wjjang2040@gmail.com

어떻게 하면 Google Drive for desktop에서 내 작업을 할 수 있습니까?

- Google Drive 파일 보기, 수정, 생성, 삭제
- Google 포토의 사진, 동영상, 앨범을 봅니다.
- 모바일 클라이언트 구성 및 실행 정보 가져오기
- 프로필 및 연락처와 같은 Google 사용자 정보 조회
- Google Drive 파일 작업 기록 조회
- Google Drive 문서 보기, 수정, 생성, 삭제

Google Drive for desktop 앱을 실행할 수 있는지 확인

명확한 정보가 이 사이트 또는 앱과 공유될 수 있습니다. 언제든지 Google 계정에서 액세스 권한을 확인하고 삭제할 수 있습니다.

Google이 데이터를 안전하게 공유하는 방법을 알아보세요.

Google Drive for desktop이 개인정보처리방침 및 서비스 약관을 확인하세요.

취소 **허용**

한국어 ▼ 도움말 개인정보처리방침 약관

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



필요한 패키지와 모듈을 불러옴

```
import matplotlib.pyplot as plt  
from tensorflow.keras.models import load_model  
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

④ 모델이 필요한 하이퍼파라미터 값 설정하기

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



하이퍼파라미터 설정

```
image_height = 150  
image_width = 150  
batch_size = 10
```

위 코드에서 학습을 위한 배치 크기(batch_size)=10,
이미지 크기(image_height=150, image_width=150)를 설정한 것을 볼 수 있음

- 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.
- 학습된 모델은 Cats vs Dogs 폴더 아래의 “**Cats_vs_Dogs_model.h5**” 파일



```
1 # 저장해 뒀던 모델을 불러온다
2
3 new_model = load_model('gdrive/My Drive/CV/Cats vs Dogs/Cats_vs_Dogs_model.h5')
4 new_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147584
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dropout (Dropout)	(None, 6272)	0
dense (Dense)	(None, 512)	3211776
dense_1 (Dense)	(None, 1)	513

Total params: 3,453,121
 Trainable params: 3,453,121
 Non-trainable params: 0

ImageDataGenerator 클래스를 통해 검증용 데이터 셋(Validation data set)에서 한번에 batch_size 만큼 이미지를 만들어내는 test_generator 객체를 생성한다.

```
# ImageDataGenerator를 통해 validation dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 test_generator 객체를 생성한다.  
test_generator = ImageDataGenerator(rescale=1./255).flow_from_directory(  
    'gdrive/My Drive/CV/Cats vs Dogs/dog-vs-cat/validation',  
    batch_size=10,  
    class_mode='binary',  
    target_size=(image_width, image_height))
```

Found 800 images belonging to 2 classes.

- 위 코드에서 검증용 데이터 셋에서 batch_size=10 만큼 이미지로 test_generator 객체가 만들어진 것을 알 수 있음
- 실행 결과 검증용 데이터 셋으로 2개의 클래스로 구성된 800장의 이미지가 있음을 알 수 있음



ImageDataGenerator 클래스를 통해 검증용 데이터 셋(Validation data set)에서 한번에 batch_size 만큼 이미지를 만들어내는 pred_generator 객체를 생성한다.

```
# ImageDataGenerator를 통해 validation dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 pred_generator 객체를 생성한다.  
pred_generator = ImageDataGenerator(rescale=1./255).flow_from_directory(  
    'gdrive/My Drive/CV/Cats vs Dogs/dog-vs-cat/validation',  
    batch_size=1,  
    class_mode='binary',  
    target_size=(image_width, image_height))
```

Found 800 images belonging to 2 classes.

- 위 코드에서 검증용 데이터 셋에서 batch_size=10 만큼 이미지로 test_generator 객체가 만들어진 것을 알 수 있음
- 실행 결과 검증용 데이터 셋으로 2개의 클래스로 구성된 800장의 이미지가 있음을 알 수 있음

다음은 테스트 데이터 셋으로 학습된 인공지능망 모델을 평가해 보자.

```
▶ # model evaluate(모델을 평가해 봅니다)
score = new_model.evaluate(test_generator, steps=800//batch_size, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1]*100)
```

```
↳ Test loss: 0.5329193472862244
Test accuracy: 73.37499856948853
```

실행 결과 테스트 데이터 셋에 대한 정확도가 73.37% 나온 것을 볼 수 있음



학습된 모델에 `pred_generator` 객체가 만들어낸 이미지로 분류 예측을 수행해 보자.

여기서는 16장 이미지를 분류 예측하고 그 결과를 `predictions` 객체에 저장함



```
# pred_generator 가 만들어낸 이미지를 우리가 만든 모델을 통해 분류 예측해 봅니다.  
predictions = model.predict(pred_generator, steps=16, verbose=1)
```



```
16/16 [=====] - 0s 6ms/step
```

학습된 모델에 `pred_generator` 객체가 만들어낸 이미지로 분류 예측을 수행해 보자.

- 다음은 `predictions` 객체의 형상과 예측결과를 확인한 것임
- 예측결과 값의 분포는 0 ~ 1사이의 실수인 것을 알 수 있음
- 즉 0 ~ 0.5 이하의 경우 'Cat', 0.5 보다 큰 값은 'Dog'로 분류됨

```
1 # predictions shape 확인
2 predictions.shape
```

```
(16, 1)
```

```
1 # 예측결과 확인
2 predictions
```

```
array([[0.1696523 ],
       [0.3645607 ],
       [0.07953901],
       [0.53780824],
       [0.6138283 ],
       [0.06508067],
       [0.02131473],
       [0.9830064 ],
       [0.50652534],
       [0.7645795 ],
       [0.98972636],
       [0.38176018],
       [0.34120196],
       [0.4365621 ],
       [0.13959974],
       [0.99547064]], dtype=float32)
```

0 ~ 0.5 이하 : Cat
0.5 보다 큰 값 : Dog



다음은 `pred_generator` 객체가 만들어 내는 **이미지의 클래스 인덱스**를 확인해 보자.

```
# pred_generator 가 만들어 내는 이미지의 클래스 인덱스가 무엇인지 확인해 본다.  
print(pred_generator.class_indices)
```

```
{'cat': 0, 'dog': 1}
```

- 실행 결과 **고양이**는 0, **개**는 1임을 알 수 있음
- 위 코드에서 `class_indices` 객체 변수의 자료구조는 딕셔너리인 것을 알 수 있음

다음은 학습된 모델이 예측한 16장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화한 것이다.

```
1 # Predict Result(pred_generator 가 만들어 낸 이미지를 우리가 만든 모델을 통해 예측한 결과를 보여줍니다.)
2 f,ax = plt.subplots(4,4,figsize=(20,20))
3
4 for i in range(len(predictions)):
5     image, label = pred_generator.next() # pred_generator 가 만들어 낸 이미지를 하나씩 꺼내온다
6     real_class = 'cat' if int(label) == 0 else 'dog' # pred_generator 가 만들어 낸 이미지의 label name 을 가져옴
7     ax[i//4,i%4].imshow(image[0]) # pred_generator 가 만들어 낸 이미지를 보여줌
8     ax[i//4,i%4].axis('off')
9
10    pred_class = 'dog' if predictions[i] > 0.5 else 'cat' # pred_generator 가 만들어 낸 이미지를 모델을 통해 예측한 결과 값의 label name
11    pred_prob = predictions[i] # pred_generator 가 만들어 낸 이미지를 모델을 통해 예측한 결과(확률값)
12    ax[i//4,i%4].set_title('REAL:' + real_class + '\nPREDICT:' + pred_class + '(' + str(pred_prob) + ')') # title 에 실제값과 모델에 의하
13
14 f.suptitle('Predict Result')
15
16 plt.show()
```

위 코드에서 predictions 객체의 원소 값이 0.5 이하이면 'Cat',
0.5보다 크면 'Dog'로 예측한 것으로 판단하고 있음

다음은 학습된 모델이 예측한 16장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화한 것이다.

예측결과를 보면 모델은 정확도 68.75%로 분류한 것을 알 수 있음

정답 : 11

오답 : 5

